

A. FOURNIER, T. LIVRAN

## TP COMPUTER GRAPHICS

### ARCBALL

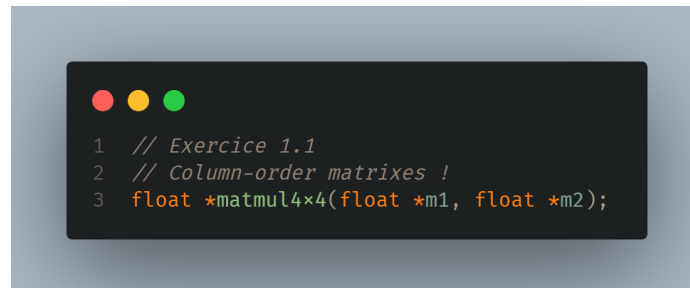
#### Partie 1 - Matrices monde

**Exercice 1.1: Implémentation d'une fonction multipliant deux matrices et renvoyant le résultat.**

Pour ce premier exercice, nous avons choisi d'écrire une fonction `float *matmul4x4(float *m1, float *m2)`.

Elle prends en paramètre pointeurs de tableaux de floats, en supposant que ces deux tableaux sont de dimensions 4x4 et effectue la multiplication matricielle. On fait évidemment attention au fait que nous sommes ici dans un mode "colonne d'abord"...

Cela donne la fonction suivante:



```
1 // Exercice 1.1
2 // Column-order matrixes !
3 float *matmul4x4(float *m1, float *m2);
```

Figure 1: final.h

Le principal désavantage de cette solution est que nous devons absolument `delete[]` la valeur retournée après utilisation.

**Exercice 1.2: remplacer vos matrices de transformation locale par une unique transformation locale-vers-monde.**

Pour cela, nous avons plusieurs modifications à apporter:

final.vert Ajout de `uniform mat4 u_world;` en tant qu'uniform, à la place de matrices individuelles de scaling, rotation...

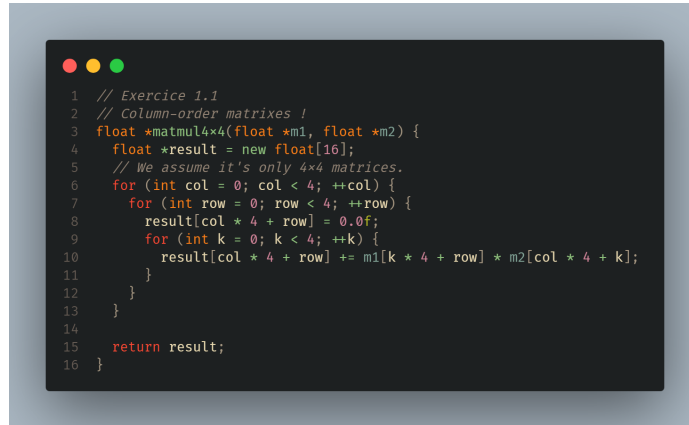


Figure 2: final.cpp

On peut donc dès à présent remplacer dans l'affectation de `gl_Position` les anciens appels et finir avec:

```
gl_Position = u_proj * u_view      * u_world * vec4(a_position,
1.0);
```

(On omet de détailler `u_proj` et `u_view` pour le moment car il s'agit de l'exercice d'après)

Le remplacement vers une matrice unique est complété du côté du GPU, du côté du CPU nous avons maintenant à calculer cette matrice, et grâce à notre helper `matmul4x4`, on peut donc procéder comme ceci:

Ce snippet de code est pris depuis la fonction `Initialize()` qui grâce au VAO & VBO permet d'alléger notre fonction `Render()` au maximum comme vu dans les TPs précédents.

Nous avons donc nos 4 matrices `Rx`, `Ry`, `Rz`, `Tr` (`S` n'est pas incluse car on n'en avait pas l'utilité au cours de ce TP), que nous combinons grâce à la méthode précédemment écrite et nous l'uploadons ensuite sur le GPU.

Comme la transformation doit valoir également pour les normales, on modifie l'affectation de `v_Normal` en:

```
v_Normal = normalize(mat3(transpose(inverse(u_world))) *
a_normal);
```

Remarque: Sachant que nous ne faisons aucune transformation sur l'objet, la matrice `World` se résume à une matrice identité `4x4`. ... Cela se vérifie

```

1 float angleX = 0.0f;
2 float angleY = 0.0f;
3 float angleZ = 0.0f;
4
5 float Rx[16] = {1,0,0,0,cosf(angleX),-sinf(angleX),0,0,sinf(angleX),cosf(angleX),0,0,0,0,1};
6
7 float Ry[16] = {cosf(angleY), 0, -sinf(angleY), 0, 0, 1, 0, 0,
8               sinf(angleY), 0, cosf(angleY), 0, 0, 0, 0, 1};
9
10 float Rz[16] = {cosf(angleZ),
11               -sinf(angleZ),
12               0,
13               0,
14               sinf(angleZ),
15               cosf(angleZ),
16               0,
17               0,
18               0,
19               0,
20               1,
21               0,
22               0,
23               0,
24               0,
25               1};
26
27 float tx = 0.f, ty = 0.f, tz = 0.f;
28 float Tr[16] = {1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, tx, ty, tz, 1};
29
30 float *temp = matmul4x4(Rx, Ry);
31 float *combined = matmul4x4(Rz, temp);
32 float *final = matmul4x4(Tr, combined);
33
34 if (g_ulocWorld >= 0) {
35     glUniformMatrix4fv(g_ulocWorld, 1, GL_FALSE, final);
36     printf("Uploaded world matrix to g_ulocWorld.\n");
37     printMatrix("World Matrix", final);
38 }

```

Figure 3: final.cpp-world

dans les logs de notre programme:

```
hex@theletter:~/e4/computer_graphics$ make final 86 ./final
-1.00000  0.00000  0.00000 -0.00000
 0.00000  1.00000 -0.00000 -0.00000
-0.00000 -0.00000 -1.00000 -5.00000
 0.00000  0.00000  0.00000  1.00000

Uploaded proj matrix to g_uLocProjProj Matrix =
 1.60948  0.00000  0.00000  0.00000
 0.00000  2.41421  0.00000  0.00000
 0.00000  0.00000 -1.00200 -0.20020
 0.00000  0.00000 -1.00000  0.00000

Uploaded world matrix to g_uLocWorld.
World Matrix =
 1.00000  0.00000  0.00000  0.00000
 0.00000  1.00000  0.00000  0.00000
 0.00000  0.00000  1.00000  0.00000
 0.00000  0.00000  0.00000  1.00000
```

Figure 4: final-output

**Exercice 2.1: codez la fonction `LookAt(vec3 position, vec3 target, vec3 up)` et passez la matrice au Vertex Shader comme View Matrix**

A partir de cet exercice la, nous avons décidé d'utiliser une librairie “boiler-plate” pour les Vecteurs3, et de l'adapter suivant les besoins. Nous avons donc utilisé Cette librairie en header-only, et ait rajouté les constructeurs dont j'avais besoin.

Par soucis de clarté nous avons également ajouté une struct `Point3` disponible dans `common/Point3.h`

Afin de coder la fonction `LookAt()`, il nous suffit de suivre les instructions. Grâce a la struct `Vector3`, la fonction est donc plutôt petite:

```

1  float *LookAt(Vector3<float> position, Vector3<float> target,
2      Vector3<float> up) {
3      Vector3<float> forward = -(target - position);
4      forward.normalize();
5
6      Vector3<float> right = up.cross(forward);
7      right.normalize();
8
9      Vector3<float> upCorrected = forward.cross(right);
10     upCorrected.normalize();
11
12     float pos_right = position * right;
13     float pos_up = position * upCorrected;
14     float pos_for = position * forward;
15
16     float *view = new float[16]{right.mX,    upCorrected.mX, forward.mX, 0,
17                                right.mY,    upCorrected.mY, forward.mY, 0,
18                                right.mZ,    upCorrected.mZ, forward.mZ, 0,
19                                -pos_right, -pos_up,    -pos_for, 1};
20
21     return view;
22 }

```

### TP à rendre - Caméra orbitale et Illumination ambiante

On considère dans un premier temps que la caméra est située sur une sphère, par conséquent, on initialise les valeurs de notre système sphérique comme ceci:

```

1  float s_cameraRadius = 6.0f;
2  float s_cameraPhi = 0.75f;
3  float s_cameraTheta = 0.35f;

```

Figure 5: final.cpp-variables

On crée également un helper

afin de traduire un Vector3 sphérique en cartésien.

Pour un rendu “interactif” Il faut donc dans un second temps mettre à jour la View Matrix grâce à notre nouvelle caméra.

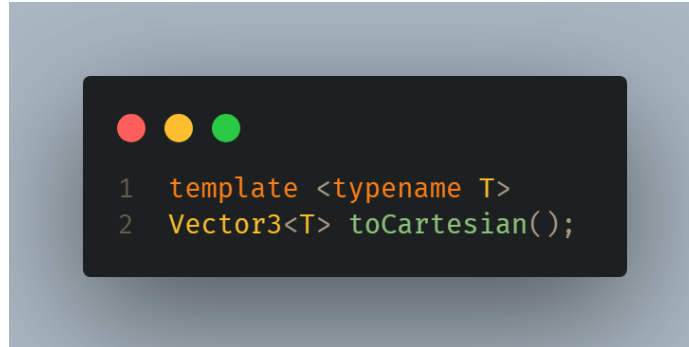


Figure 6: final.h

Pour cela, nous avons écrit un helper `updateViewMatrix()` qui est appelé à chaque `Render()` pour inclure les changements induits par les mouvements de souris/molette.

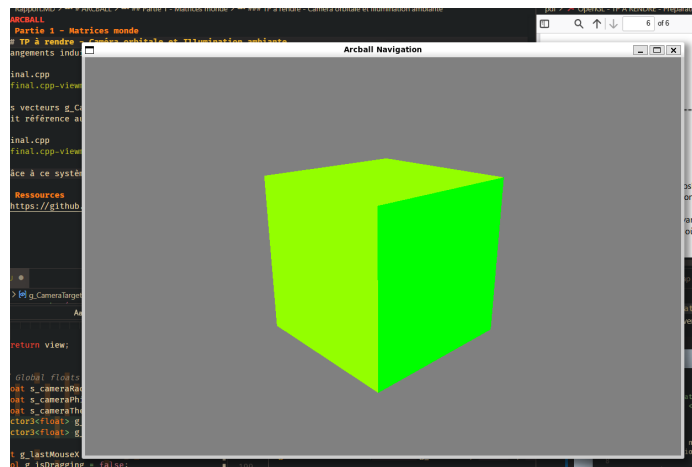


Figure 7: final.cpp-viewmatrix

Les vecteurs `g_CameraTarget` et `g_UpVector` sont la position de notre objet (ici un cube) et `g_UpVector` fait référence au vecteur unitaire `u_y`:



Figure 8: final.cpp-viewmatrix-init



Grâce à ce système, nous avons donc une vue fonctionnelle de notre cube, afin d'ajouter les fonctionnalités de mouvement requise, nous devons donc nous reporter à cette page afin d'ajouter les différentes composantes requises.

Dans un premier temps, nous avons du basculer l'entièreté de notre programme sur GLFW. En effet, nous avons commencé à l'écrire avec GLUT.

Les lignes importantes ici sont les lignes 29-31 aubsu que 41.

Nous enregistrons des callbacks pour 3 évènements: - Lorsque la souris bouge (angle theta et phi affectés) - Lorsque la molette bouge (Radius affecté) - Lorsque l'on clique avec le clic gauche (éviter les mouvements intempestifs, clause barrière dans le handler `cursor_position_callback`)

```

1 int main(int argc, char **argv) {
2     if (glfwInit() != GLFW_TRUE) {
3         fprintf(stderr, "Error initializing GLFW");
4         return EXIT_FAILURE;
5     }
6
7     // Hints suggested by AI
8     glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 2);
9     glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 1);
10    glfwWindowHint(GLFW_DEPTH_BITS, 24);
11
12    g_Window = glfwCreateWindow(W_WIDTH, W_HEIGHT, "Arcball Navigation", nullptr,
13                                nullptr);
14
15    if (g_Window == nullptr) {
16        fprintf(stderr, "Error initializing Window");
17        glfwTerminate();
18        return EXIT_FAILURE;
19    }
20
21    glfwMakeContextCurrent(g_Window);
22    glfwSwapInterval(1);
23
24    if (GLEW_OK != glewInit()) {
25        fprintf(stderr, "Error initializing GLEW");
26        return EXIT_FAILURE;
27    }
28
29    glfwSetCursorPosCallback(g_Window, cursor_position_callback);
30    glfwSetMouseButtonCallback(g_Window, mouse_button_callback);
31    glfwSetScrollCallback(g_Window, scroll_callback);
32
33    if (!Initialize()) {
34        fprintf(stderr, "Error Initializing program");
35        return EXIT_FAILURE;
36    }
37
38    while (!glfwWindowShouldClose(g_Window)) {
39        Render();
40        glfwSwapBuffers(g_Window);
41        glfwPollEvents();
42    }
43
44    Terminate();
45
46    return EXIT_SUCCESS;
47 }

```

Figure 9: final.cpp-main



```
1 static void cursor_position_callback(GLFWwindow *window, double xpos,
2                                     double ypos) {
3     (void)window;
4
5     int x = (int)xpos;
6     int y = (int)ypos;
7
8     if (lg_isDragging) {
9         g_lastMouseX = x;
10        g_lastMouseY = y;
11        return;
12    }
13
14    int dx = x - g_lastMouseX;
15    int dy = y - g_lastMouseY;
16
17    g_lastMouseX = x;
18    g_lastMouseY = y;
19
20    // Sensitivity needed otherwise camera just doesn't move
21    const float sensitivity = 0.01f;
22
23    s_cameraPhi += (float)dx * sensitivity;
24    s_cameraTheta -= (float)dy * sensitivity;
25
26    // Clamping near "bornes" adds issues, add a small offset:
27    float off = 0.01f;
28
29    s_cameraPhi = std::clamp<float>(s_cameraPhi, -M_PI + off, M_PI - off);
30    s_cameraTheta =
31        std::clamp<float>(s_cameraTheta, -M_PI / 2 + off, M_PI / 2 - off);
32 }
```

Ci-dessous les 3 callbacks:

Remarques: - (void)window sert à supprimer un warning concernant les paramètres non-utilisés - On utilise deux variables “globales” `g_lastMouseX` et `g_lastMouseY` afin d’enregistrer l’état précédent et calculer un `deltaX` et `deltaY`. - après tests, il s’est avéré qu’une variable `sensitivity` était nécessaire pour baisser la valeur de `s_cameraPhi` & `s_cameraTheta` car sinon on ne voyait tout simplement pas le mouvement, le clamping marchait “trop” bien. - l’offset `off` sert à éviter les effets de bords aux bornes  $\pi$ ,  $-\pi$  et  $\pi/2$ ,  $-\pi/2$  car nous avons observé des glitches graphiques quand le mouvement arrivait dans ces zones là.

```
1 void scroll_callback(GLFWwindow *window, double xoffset, double yoffset) {
2     (void)xoffset;
3     (void)window;
4
5     s_cameraRadius = std::max(5.f, s_cameraRadius - (float)yoffset * 0.3f);
6 }
```

Figure 10: final.cpp-scroll-callback

Remarques: - On veut éviter de “traverser” l’objet donc on s’assure d’avoir un minimum de 5 unités entre la caméra et le target, qui est (0,0,0). Cela revient à Translater l’objet de quelque unités en arrière (axe -Z) comme vu dans les TP précédents ! - On est obligé d’appliquer un facteur 0.3f car sinon la molette zoom beaucoup trop et on a pas d’entre deux..

A screenshot of a code editor window with a dark background and light-colored text. The code is C++ and defines a static void function named mouse\_button\_callback. The function takes four parameters: GLFWwindow \*window, int button, int action, and int mods. It first checks if the button is not GLFW\_MOUSE\_BUTTON\_LEFT; if so, it returns. Then, it sets g\_isDragging to true if the action is GLFW\_PRESS. If g\_isDragging is true, it declares double variables x and y, initializes them to 0.0, and then calls glfwGetCursorPos to get the current cursor position. Finally, it updates g\_lastMouseX and g\_lastMouseY with the current x and y values. The code is numbered from 1 to 18.

```
1 static void mouse_button_callback(GLFWwindow *window, int button, int action,
2                                     int mods) {
3     (void)mods;
4
5     if (button != GLFW_MOUSE_BUTTON_LEFT) {
6         return;
7     }
8
9     g_isDragging = (action == GLFW_PRESS);
10
11     if (g_isDragging) {
12         double x = 0.0;
13         double y = 0.0;
14         glfwGetCursorPos(window, &x, &y);
15         g_lastMouseX = (int)x;
16         g_lastMouseY = (int)y;
17     }
18 }
```

Figure 11: final.cpp-button-callback

Remarques: - Ce callback sert uniquement à éviter d’avoir trop de mouvements intempestifs.. Il a été suggéré par l’IA afin de rendre les tests plus simples.

### Résultats finaux:

#### Vidéo disponible dans le dossier: test-wa.mkv

Afin d’obtenir un résultat plus “joli”, nous avons appliqué ce que nous avons appris lors du TP de l’illumination. Pour atteindre ce résultat, nous avons du par ailleurs transformer les shaders.

Nous passons donc de la version 120 à 430. Nous avons été aidé par les ressources spécifiées dans **Ressources** pour cette migration.

#### Vidéo avec illumination (animation sans navigation) disponible dans le dossier: test-wa-ill.mkv

Nous avons également décidé d’ajouter un handler pour la touche espace, afin de mettre en pause l’animation que j’effectue. Ce qui rend la navigation en mode “arcball” plus facile à voir.

En dernier lieu, un test avec des textures custom a été réalisé ci-dessous:

**Vidéo avec illumination, texture, navigation, pause disponible dans le dossier: test-wa-ill-tex-nav-pause.mkv**

Tout le travail a été réalisé sur Windows 11 avec WSL 2 (Ubuntu).

## **Ressources**

- <https://github.com/Gen346/Vector3>
- <https://github.com/mattdesl/lwjgl-basics/wiki/GLSL-Versions>